

Advanced Level Quantitative Finance Projects

A Practical Guide for Aspiring Quants

Amit Kumar Jha , Quant Analyst at UBS

November 20, 2025

```

43         vol_interp = interp1d(self.maturity_grid, vol_factor, kind='
         cubic')
44         self.vol_functions.append(vol_interp)
45
46     return self.vol_functions
47
48 def compute_drift(self, t_idx, vol_factors):
49     """Compute HJM drift term at time step t_idx"""
50     M = len(self.maturity_grid)
51     drift = np.zeros(M)
52
53     for k, vol in enumerate(vol_factors):
54         # Current volatilities for all maturities
55         sigma_tT = vol(self.maturity_grid - self.time_grid[t_idx])
56
57         # Compute integral using numerical quadrature
58         integral = np.zeros(M)
59         for j in range(M):
60             # Integrate sigma_k(t, u) from t to T_j
61             u_grid = np.linspace(self.time_grid[t_idx], self.
                maturity_grid[j], 50)
62             sigma_grid = vol(u_grid - self.time_grid[t_idx])
63             integral[j] = np.trapz(sigma_grid, u_grid)
64
65         drift += sigma_tT * integral
66
67     return drift
68
69 def simulate(self, n_paths=50000, n_factors=3):
70     """Monte Carlo simulation of forward rate curve"""
71     N = len(self.time_grid) - 1
72     M = len(self.maturity_grid)
73
74     # Initialize paths
75     paths = np.zeros((n_paths, N+1, M))
76     paths[:, 0, :] = self.forward_curve
77
78     # Store volatility functions
79     vol_factors = self.vol_functions[:n_factors]
80
81     for i in range(N):
82         print(f"Simulating time step {i+1}/{N}")
83
84         # Compute drift
85         drift = self.compute_drift(i, vol_factors)
86
87         # Random shocks
88         Z = np.random.randn(n_paths, n_factors)

```

```

122 # XVA engine
123 xva_engine = XVAEngine(portfolio, discount_curve, funding_curve)
124
125 # Compute metrics
126 xva_metrics = xva_engine.compute_xva_metrics(T=5.0, n_paths=100000)
127
128 print("\nXVA Breakdown:")
129 for metric, value in xva_metrics.items():
130     print(f"{metric}: ${value/1e6:.2f}M")
131
132 # Visualize exposure profile
133 epe_profile, _, t_grid = xva_engine.simulate_portfolio_mtm(5.0,
134     100000, 252)
135
136 plt.figure(figsize=(12, 8))
137 plt.plot(t_grid, epe_profile / 1e6, 'b-', linewidth=2)
138 plt.fill_between(t_grid, 0, epe_profile / 1e6, alpha=0.3)
139 plt.xlabel('Time (years)')
140 plt.ylabel('Expected Positive Exposure ($M)')
141 plt.title('Portfolio EPE Profile for XVA')
142 plt.grid(True)
143 plt.savefig('xva_epe_profile.png')
144 plt.show()

```

C++ Code (XVA Calculator)

```

1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <random>
5 #include <Eigen/Dense>
6 #include <unsupported/Eigen/FFT>
7
8 class XVAEngine {
9 private:
10     struct Trade {
11         std::string type;
12         double notional;
13         double tenor;
14         double fixed_rate;
15     };
16
17     std::vector<Trade> portfolio_;
18     std::function<double(double)> discount_curve_;
19     std::function<double(double)> funding_curve_;
20     double r_ = 0.05;
21

```

```

22 public:
23     XVAEngine(const std::vector<Trade>& portfolio,
24               std::function<double(double)> discount_curve,
25               std::function<double(double)> funding_curve)
26     : portfolio_(portfolio), discount_curve_(discount_curve),
27       funding_curve_(funding_curve) {}
28
29 struct ExposureProfile {
30     Eigen::VectorXd epe;
31     Eigen::VectorXd ene;
32     Eigen::VectorXd time_grid;
33 };
34
35 // Simulate portfolio MtM
36 ExposureProfile simulatePortfolioMTM(double T, int n_paths, int
37 n_steps) {
38     double dt = T / n_steps;
39     Eigen::VectorXd t_grid = Eigen::VectorXd::LinSpaced(n_steps + 1,
40     0, T);
41
42     // Simulate risk factor (simplified)
43     Eigen::MatrixXd r_paths = Eigen::MatrixXd::Zero(n_paths, n_steps
44     + 1);
45     r_paths.col(0).setConstant(0.03);
46
47     std::mt19937 rng(42);
48     std::normal_distribution<> normal(0.0, 1.0);
49
50     for (int i = 0; i < n_steps; ++i) {
51         Eigen::VectorXd Z(n_paths);
52         for (int j = 0; j < n_paths; ++j) Z[j] = normal(rng);
53
54         r_paths.col(i+1) = r_paths.col(i) + 0.1 * (0.03 - r_paths.col
55         (i)) * dt +
56
57             0.01 * std::sqrt(dt) * Z;
58     }
59
60     // Portfolio MtM
61     double dv01 = 1e4;
62     Eigen::MatrixXd portfolio_values = dv01 * (r_paths - 0.03 * Eigen
63     ::MatrixXd::Ones(n_paths, n_steps + 1));
64
65     // Close-Out Amount (10-day MPoR)
66     Eigen::MatrixXd coa = Eigen::MatrixXd::Zero(n_paths, n_steps + 1)
67     ;
68     for (int i = 0; i < n_steps - 10; ++i) {
69         coa.col(i) = r_paths.block(0, i, n_paths, 10).rowwise().mean
70         ();
71     }
72 }

```